Autonomous SOC Platform & System Behavior Analytics (SBA): Comprehensive Technical Architecture and Operational Report

1. Executive Summary & Architectural Vision

Modern Security Operations Centers (SOCs) face unprecedented telemetry volume, characterized by alert fatigue, expanding attack surfaces, and sophisticated multi-stage threats that easily bypass static signature heuristics. Traditional SOC workflows rely heavily on Tier-1 and Tier-2 human analysts manually pivoting across disparate tools—firewalls, Network Intrusion Detection Systems (NIDS), Endpoint Detection and Response (EDR/XDR) platforms, and Threat Intelligence databases. This creates high Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR) metrics, while increasing the risk of analyst burnout and missed true positives.

This project architects, implements, and validates an **Autonomous Security Operations Center (Autonomous SOC)**. The platform pairs a high-performance, deterministic network and host-level monitoring lab with a dual-engine analytical framework:

1. **The Autonomous Cognitive Agent Platform:** An asynchronous, event-driven orchestration system built on FastAPI, LangGraph, and Google Gemini 3.1 Flash Lite. It autonomously ingests, parses, validates, enriches, and investigates endpoint and network security incidents via a ReAct (Reasoning and Acting) execution loop.

2. **The System Behavior Analytics (SBA) Microservice:** An unsupervised Machine Learning (PyOD Isolation Forest) and Explainable AI (SHAP TreeExplainer) microservice that operates alongside rule-based alerts. It continuously parses raw, unstructured telemetry streams using Drain3 template mining, computes host-level baseline behavior across sliding time windows, identifies subtle contextual anomalies, and outputs mathematically justified situational awareness directly into the SIEM and agent decision pipelines.

2. IT, Enterprise Network & Lab Infrastructure Engineering

The underlying infrastructure provides full-packet capture, deep network inspection, granular endpoint instrumentation, and multi-tier log routing across isolated virtual networks.

2.1 Virtual Network Topology & Subnet Design

The enterprise testing environment is segmented across three primary interfaces managed by an enterprise pfSense Community Edition Firewall/Router:

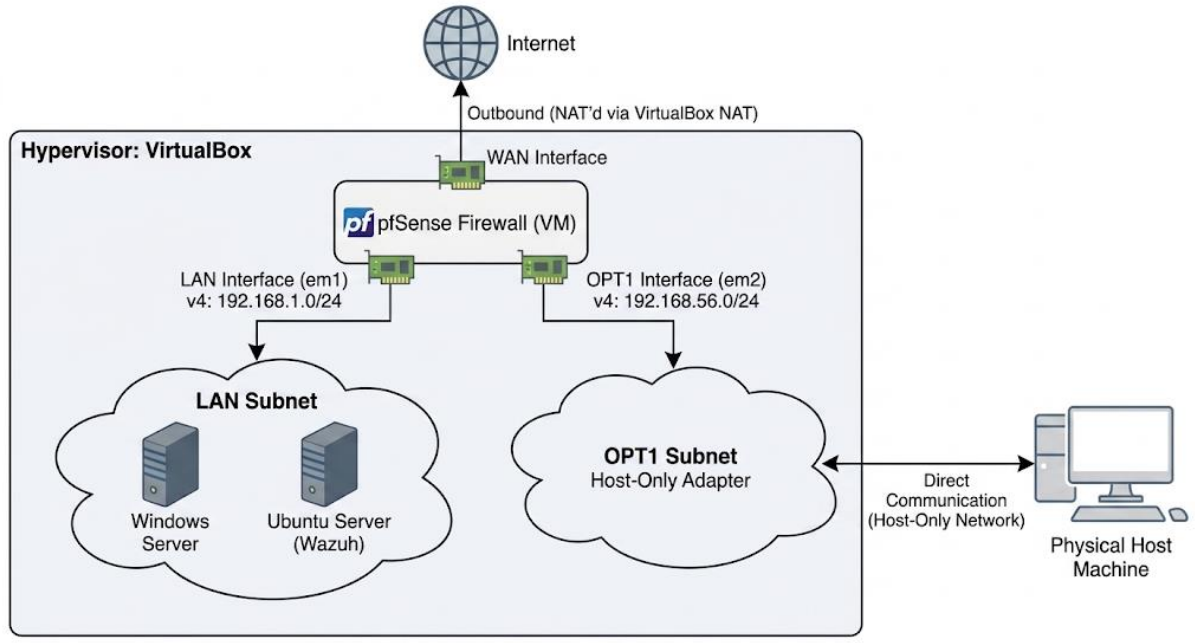


Figure 1 Network topology

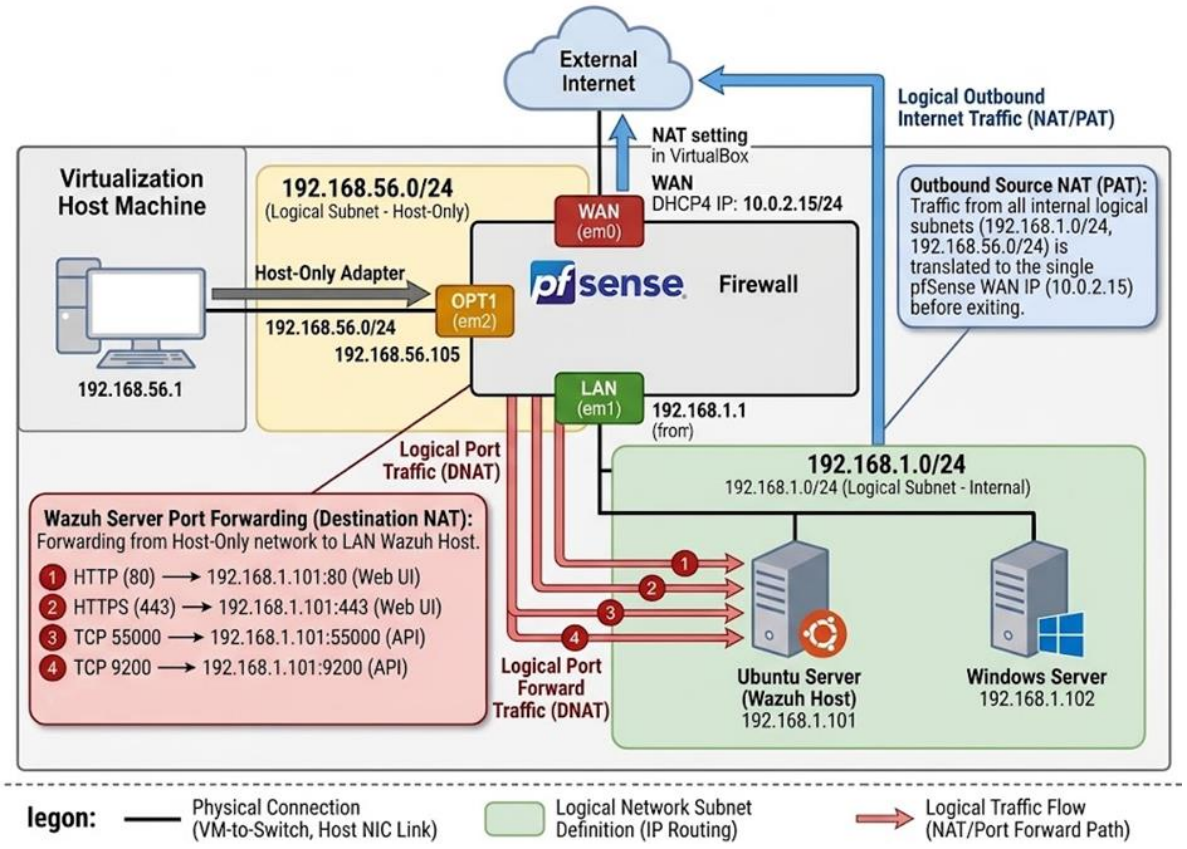


Figure 2 Logical topology with port forwarding

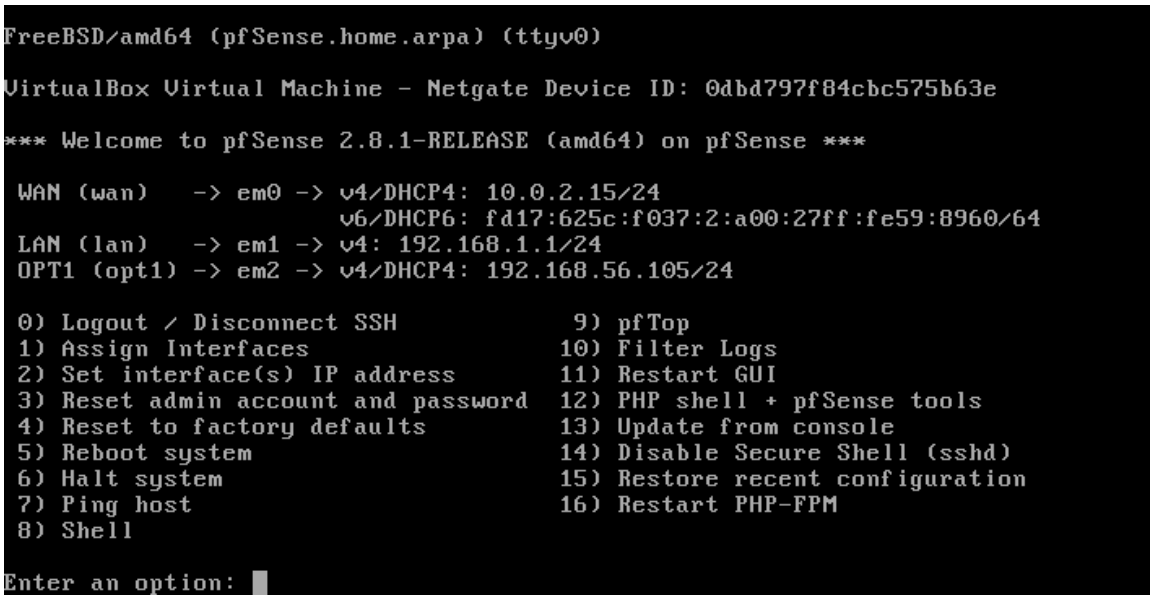


Figure 3 Pfsense interfaces configuration

1. **WAN Interface (`em0`):**
    - **Configuration:** Configured via IPv4 DHCP to receive upstream connectivity (`10.0.2.15/24`, Default Gateway `10.0.2.2`)
    - **Role:** Handles outbound telemetry, threat intelligence API resolutions, and sandboxed malware external command-and-control (C2) communication.

2. **LAN Interface (`em1`):**
    - **Configuration:** Static IPv4 `192.168.1.1/24`.
    - **Role:** Dedicated internal broadcast domain hoting enterprise endpoints and internal telemetry aggregators.
    - **Monitored Endpoints:**
        - **Windows Server 2019 Datacenter (`WIN-5RJ7UQ2G3GE`):** Static IP `192.168.1.100`, designated as **Wazuh Agent** `001`.
        - **Ubuntu Linux Enterprise SIEM (`ubuntuserver`):** Static IP `192.168.1.101`, hosting the Wazuh Manager, OpenSearch Indexer, and Filebeat pipeline.

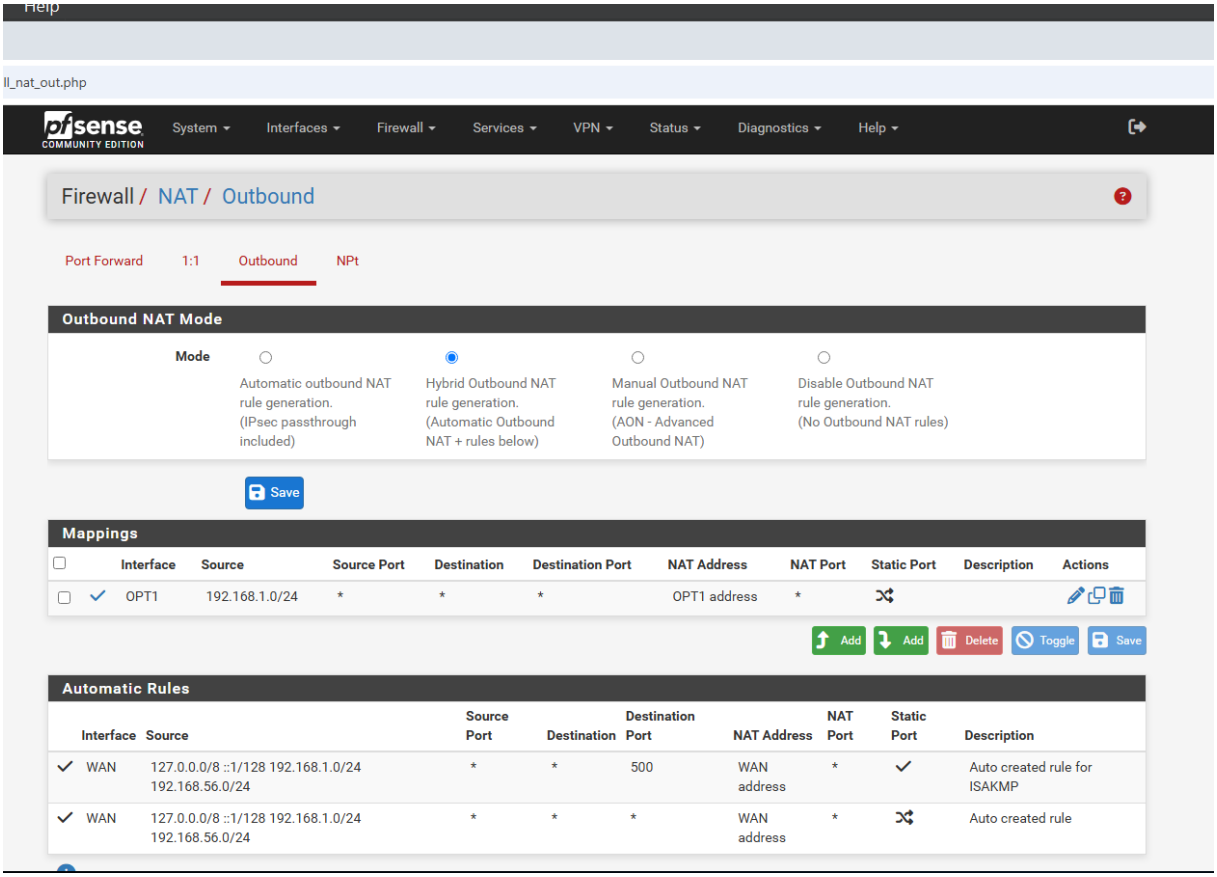
3. **OPT1 Host-Only Interface (`em2`):**
    - **Configuration:** Static IPv4 `192.168.56.100/24`.
    - **Role:** Out-of-band management interface routing between the host development hypervisor and virtualized internal nodes.

2.2 pfSense Firewall Configuration, Routing, and NAT Policies

- **Outbound Network Address Translation (NAT)**

The firewall operates in **Hybrid Outbound NAT** mode to guarantee that all internal assets within the `192.168.1.0/24` subnet can resolve domain names and establish outbound connections (e.g., threat intelligence feeds, updates, or adversary infrastructure) while masking internal IP schemes behind the firewall's WAN interface (`10.0.2.15`):

Source: 192.168.1.0/24 ⟶ NAT Address: WAN address (10.0.2.15) ⟶ Destination: ∗



- **Inbound Port Forwarding (NAT Mapping)**

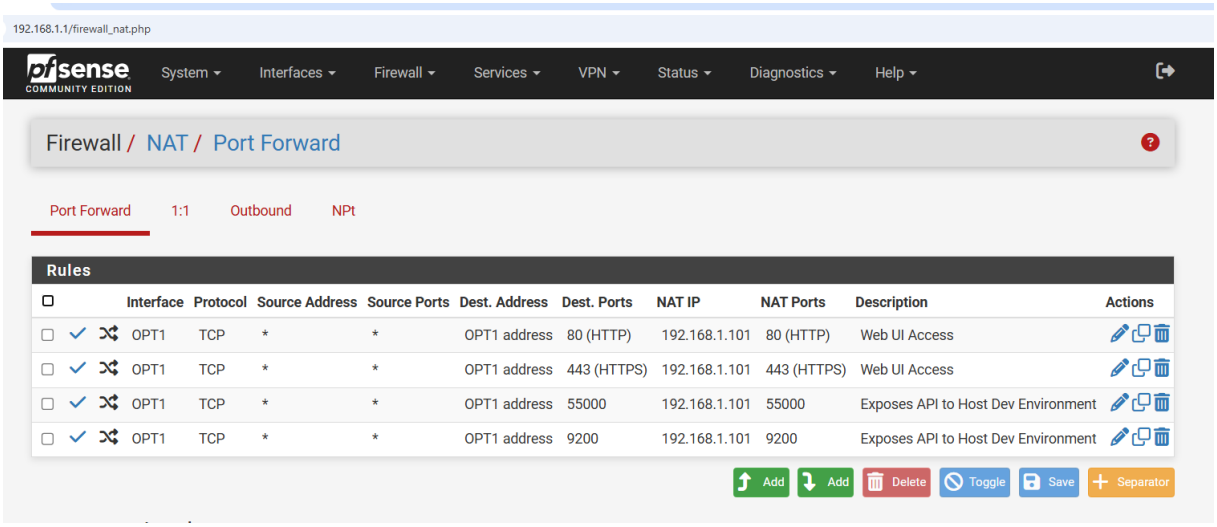


Figure 4 Port Forwarding settings

To expose the internal monitoring stack on `192.168.1.101` to the host development workstation over the `OPT1` host-only bridge (`192.168.56.100`), deterministic port-forwarding rules are applied:

Interface	Protocol	Source	Dest. Address	Dest. Port	NAT Target IP	NAT Target Port	Description
OPT1	TCP	*	`OPT1 address`	80	`192.168.1.101`	80 (HTTP)	Wazuh Dashboard Web UI Redirection
OPT1	TCP	*	`OPT1 address`	443	`192.168.1.101`	443 (HTTPS)	Wazuh Dashboard Secure SSL Web UI
OPT1	TCP	*	`OPT1 address`	55000	`192.168.1.101`	55000	Wazuh Core Management RESTful API
OPT1	TCP	*	`OPT1 address`	9200	`192.168.1.101`	9200	OpenSearch Cluster Ingestion / REST DSL

Table 1 Inbound Port Forwarding (NAT Mapping)

Firewall Filter Rules (Packet Filtering Engine)

- **OPT1 Ruleset:** An explicit `pass` rule (`Allow all inbound on OPT1`) permits bidirectional developer traffic between the local analysis machine (`192.168.56.1/24`) and the virtual lab environment.
- **LAN Ruleset:** Configured to permit internal broadcast and unicast communication across nodes (`192.168.1.0/24`), while forcing all egress default gateway traffic through pfSense's packet inspection engines.

2.3 Network Intrusion Detection System (NIDS): Suricata Configuration

Suricata runs directly on the pfSense perimeter gateway to provide inline/promiscuous deep packet inspection (DPI) of all inter-VLAN and egress/ingress WAN flows.

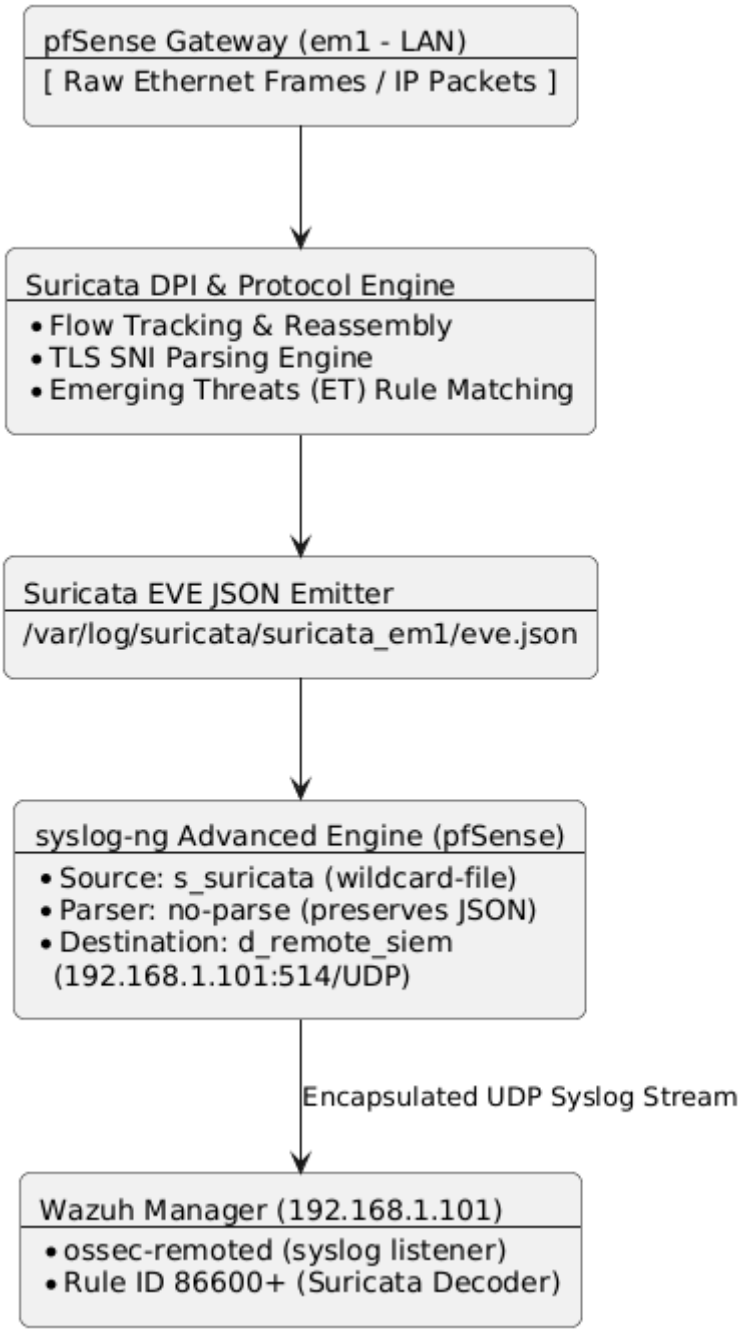
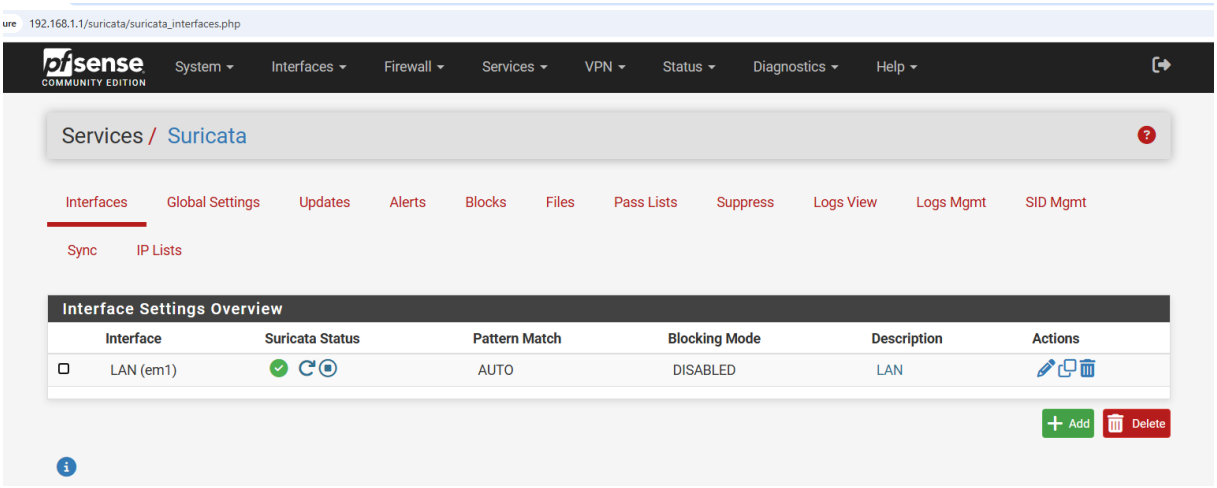


Figure 5 suricata ids logs flows

Interface Binding & Operational Settings

- **Interface:** Attached to `LAN (em1)` in promiscuous capture mode, giving it full visibility into un-NATted internal IP addresses (`192.168.1.100`, `192.168.1.101`).
- **Pattern Matcher:** Hardware-optimized `AUTO` (Hyperscan/Aho-Corasick hybrid pattern matching).



- **Rule Management:** Ingests the **Emerging Threats (ET) Open Ruleset**, specifically tracking:
    - *ET MALWARE*: Malware Command and Control channels, known malware TLS Server Name Indication (SNI) signatures, and dynamic DNS callbacks.
    - *ET EXPLOIT*: Active vulnerability exploits, buffer overflows, and remote code execution payloads.
    - *ET SCAN*: Port reconnaissance, vulnerability scans, and mass-can discovery traffic.

EVE JSON Output Engine

Suricata's unified Extensible Event Format (EVE JSON) is enabled (`eve.json`) at `/var/log/suricata/suricata_em1/eve.json`. This generates structured, single-line JSON log documents capturing packet context:

- **`EVE Log Alerts:`** Fully enabled with packet payloads (`Payload Data Formats: HEX/ASCII`), packet decoders, and application-layer metadata.
- **`App-Layer Metadata Enabled:`**
    - **TLS Handshakes:** Logs complete client certificates, server certificates, cipher suites, TLS versions, and crucially, **TLS SNI (`tls.sni`)** records to expose encrypted HTTPS C2 communication targets.
    - **HTTP Logging:** Ingests URI parameters, User-Agent headers, Host headers, and response codes.
    - **DNS Engine:** Ingests query names, record types (`A`, `AAAA`, `CNAME`, `TXT`), and responses.
    - **Flow Tracking:** Ingests TCP flags, flow timers, and aggregate byte counters.

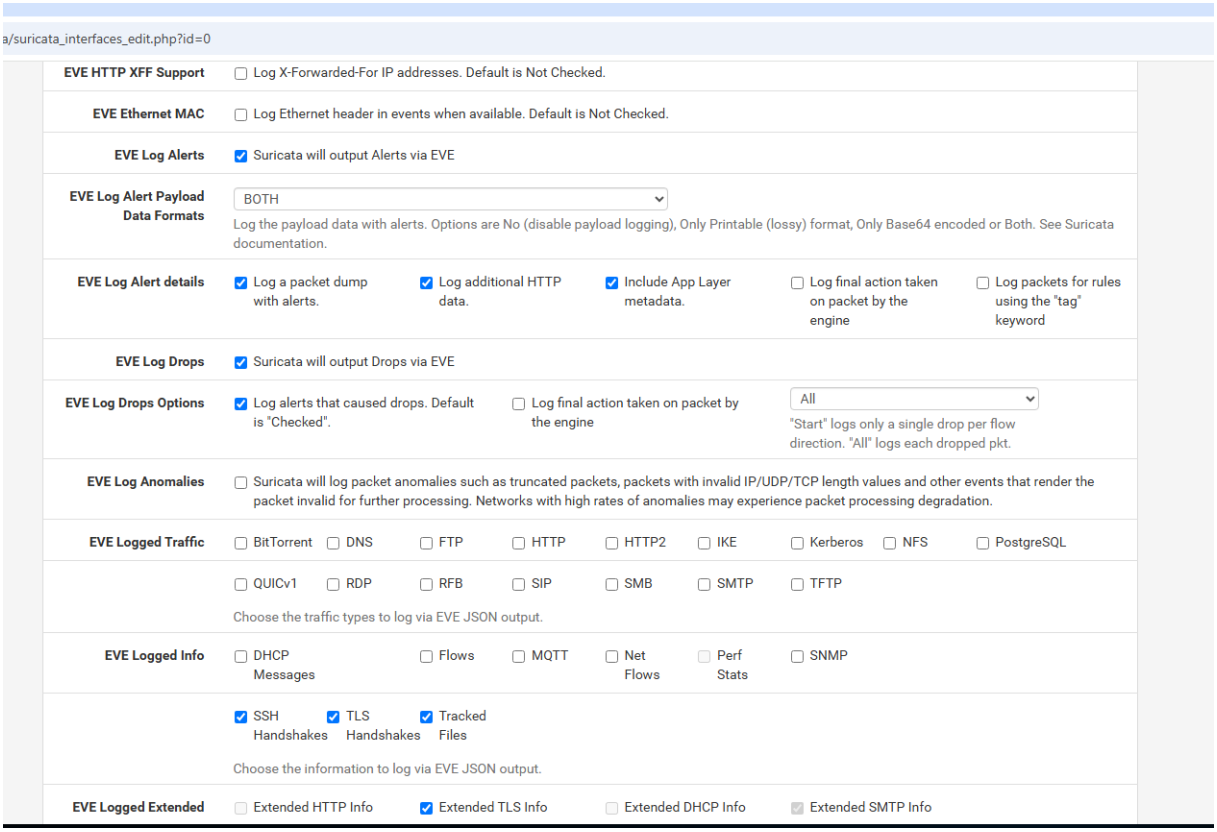
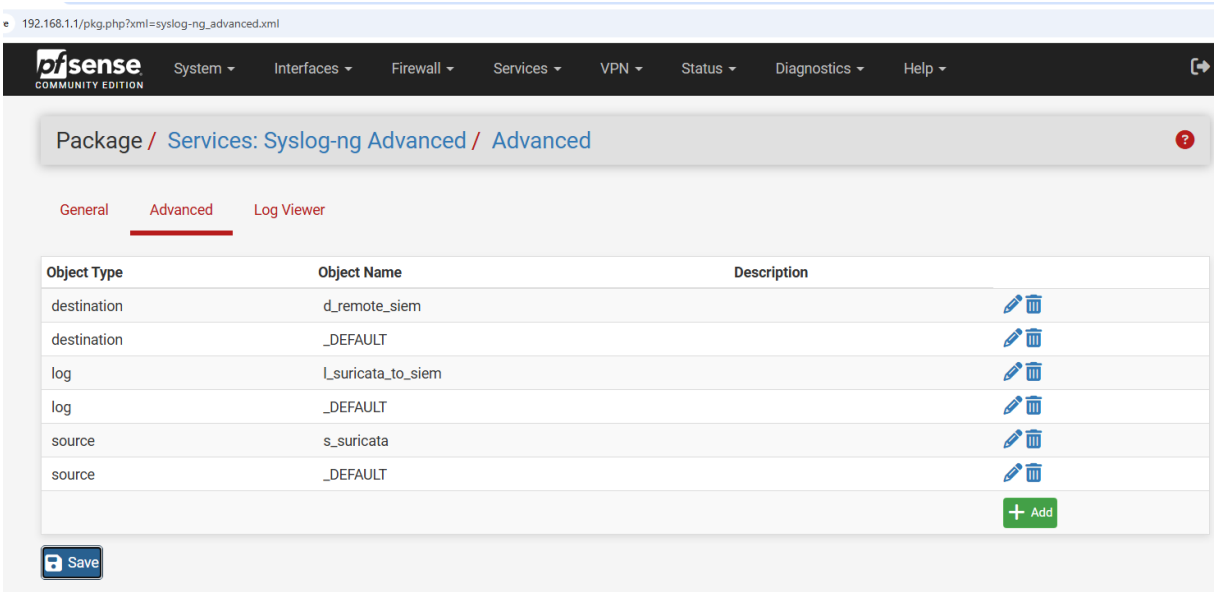


Figure 6 suricata IDS settings

2.4 High-Performance Log Routing: syslog-ng on pfSense

To transmit Suricata's high-velocity JSON events across the network without schema distortion or kernel-space buffer drops, the **syslog-ng Advanced Engine** is deployed on pfSense.



1. Source Definition (`s_suricata`)

syslog-ng tails the live EVE JSON file directly from the filesystem using a non-blocking `wildcard-file` source:

```
source s_suricata {
    file("/var/log/suricata/suricata_em1/eve.json"
         flags(no-parse)
         follow-freq(1)
         program-override("suricata")
    );
};
```

Engineering Detail: The `flags(no-parse)` directive is critical. It prevents the local syslog daemon from treating Suricata's JSON payload as standard RFC 3164/5424 structured syslog text, preventing log truncations and preserving JSON string integrity.

2. Destination Definition (`d_remote_siem`)
Configured to transmit logs directly to the Ubuntu SIEM Manager via UDP transport:

```
destination d_remote_siem {
    syslog("192.168.1.101"
           port(514)
           transport("udp")
           template("${MESSAGE}\n")
    );
};
```

3. Log Route Mapping (`log { ... }`)
Binds the input file-monitoring stream to the network socket destination:

```
log {
    source(s_suricata);
    destination(d_remote_siem);
};
```

2.5 SIEM & Log Aggregation Architecture: Wazuh Manager & OpenSearch

The telemetry collection node runs on Ubuntu Linux 22.04 LTS (`192.168.1.101`), executing a hardened, multi-tiered log analysis and storage architecture.

Wazuh Server Ingestion Configuration (`/var/ossec/etc/ossec.conf`)

1. Secure Agent-to-Server Transport

```
<remote>
    <connection>secure</connection>
    <port>1514</port>
    <protocol>tcp</protocol>
    <queue_size>131072</queue_size>
</remote>
```

Provides TLS-encrypted, authenticated agent communications for internal endpoints (Windows Server Agent `001`) on port `1514/TCP`, backed by a memory buffer queue size of `131,072` events to handle traffic spikes without data loss.

2. Network Syslog Listener

```
<remote>
    <connection>syslog</connection>
    <port>514</port>
    <protocol>udp</protocol>
    <allowed-ips>192.168.1.0/24</allowed-ips>
</remote>
```

Exposes an asynchronous UDP syslog receiver on port `514`, bound to the `192.168.1.0/24` subnet. It ingests Suricata EVE JSON logs emitted by pfSense. Wazuh's internal decoding engine routes incoming raw JSON strings through `/var/ossec/ruleset/decoders/0385-suricata_decoders.xml`, normalizing network threat observables into the SIEM core.

3. Automated Webhook Forwarding Integration

```
<integration>
    <name>custom-soc</name>
    <hook_url>http://192.168.56.1:8000/webhook/wazuh</hook_url>
    <level>3</level>
    <alert_format>json</alert_format>
</integration>
```

Binds the Wazuh analysis daemon (`ossec-analysisd`) to the custom Autonomous SOC FastAPI application endpoint (`[http://192.168.56.1:8000/webhook/wazuh](http://192.168.56.1:8000/webhook/wazuh)`). Any correlated rule with a severity level ≥ 3 triggers an asynchronous JSON HTTP POST request containing the alert payload, initiating the cognitive investigation pipeline.

4. Alert Logging Matrix

```
<global>
    <jsonout_output>yes</jsonout_output>
    <alerts_log>yes</alerts_log>
    <logall>yes</logall>
    <logall_json>yes</logall_json>
</global>

<alerts>
    <log_alert_level>3</log_alert_level>
    <email_alert_level>12</email_alert_level>
</alerts>
```

Forces Wazuh to write structured outputs to `/var/ossec/logs/alerts/alerts.json`. By setting `<logall>` and `<logall_json>` to `yes`, all telemetry—including non-alerting benign baseline system activity—is written to `/var/ossec/logs/archives/archives.json`. This provides the comprehensive raw training dataset required by the System Behavior Analytics (SBA) microservice.



Figure 7 tcp dump in the ubuntu machine showing the suricata logs being recived

2.6 Filebeat Telemetry Pipeline & OpenSearch Storage Layer

To index millions of unstructured and structured events into OpenSearch (`wazuh-indexer`), Filebeat is deployed as an asynchronous log shipper (`/etc/filebeat/filebeat.yml`).

```
output.elasticsearch:
  hosts: ["https://127.0.0.1:9200"]
  username: "admin"
  password: "${INDEXER_PASSWORD}"
  ssl.certificate_authorities: ["/etc/filebeat/certs/root-ca.pem"]
  ssl.certificate: "/etc/filebeat/certs/ubuntuserver.pem"
  ssl.key: "/etc/filebeat/certs/ubuntuserver-key.pem"
  setup.template.json.enabled: true
  setup.template.json.path: "/etc/filebeat/wazuh-template.json"
  setup.template.json.name: "wazuh"
  setup.ilm.enabled: false

filebeat.modules:
  - module: wazuh
    alerts:
      enabled: true
    archives:
      enabled: true
```

- **Dual Module Ingestion:** Filebeat splits data into two distinct OpenSearch index families:
    1. `wazuh-alerts-4.x-*`: Ingests high-fidelity security events (rule level ≥ 3) for live SIEM search and dashboarding.
    2. `wazuh-archives-4.x-*`: Ingests the continuous, raw system event stream across all hosts for ML baseline training and historical DSL queries.
- **Index Template Sharding:** Ingests strict index mappings via `wazuh-template.json`, enforcing strict data types on nested properties (e.g., `data.win.eventdata.processId`, `data.win.system.eventID`, `data.srcip`).

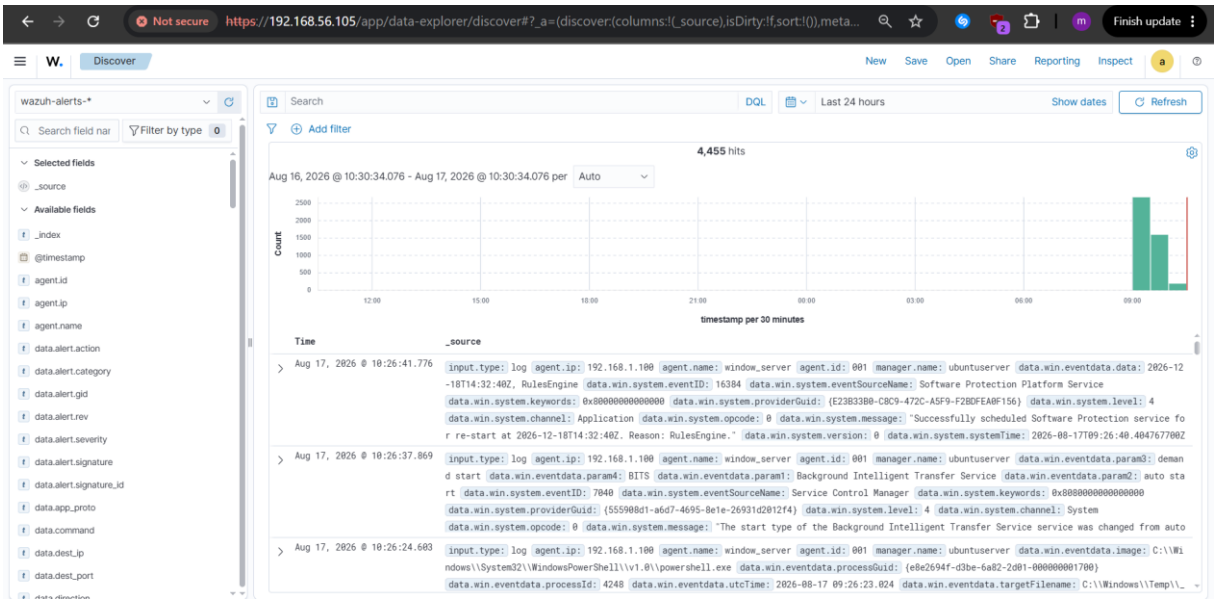


Figure 8 wazuh dashbord with the wazuh-alert- index*

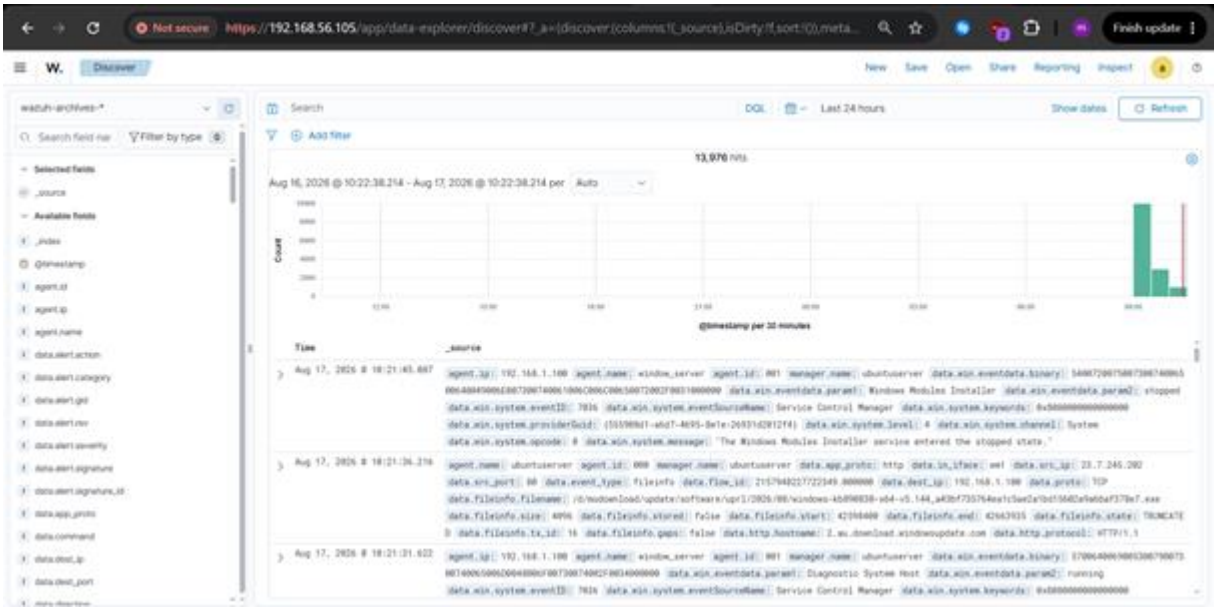
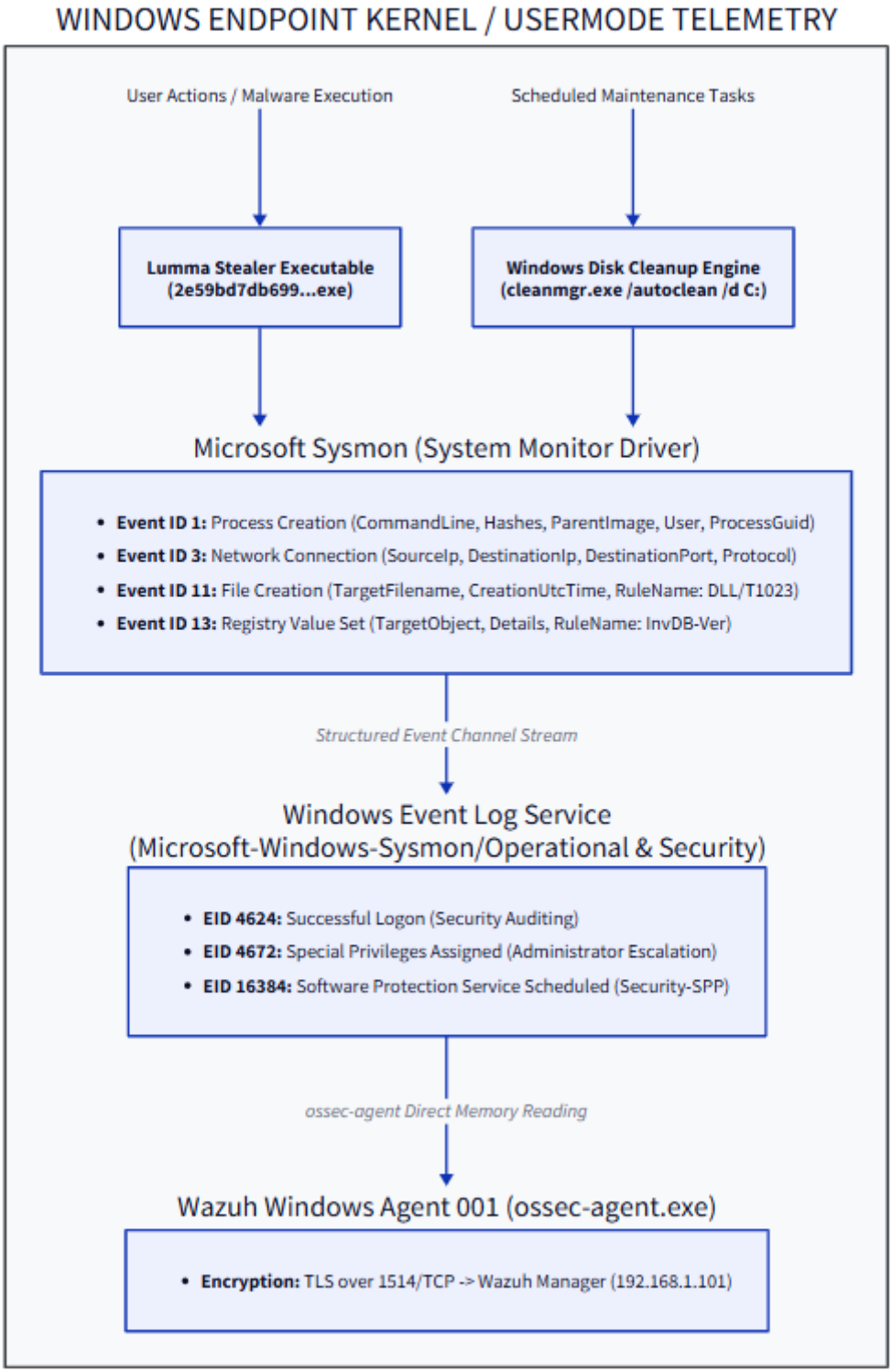


Figure 9 wazuh dashbord with the wazuh-archives- index*

2.7 Endpoint Instrumentation: Windows Server 2019 & Microsoft Sysmon

The Windows Server 2019 Datacenter machine (`WIN-5RJ7UQ2G3GE` / Agent `001`) runs Microsoft System Monitor (Sysmon v13) alongside the native Windows Event Log channel. This configuration captures host-level process and kernel activity:



Monitored Event Channel Types

1. Sysmon Event ID 1 (Process Create)
Tracks process execution, parent-child relationships, command-line arguments, user contexts, and cryptographic hashes:

- `data.win.eventdata.image`: Path to the executed binary (e.g., `C:\Windows\system32\cleanmgr.exe` or desktop executables).
- `data.win.eventdata.parentImage`: Path of the spawning parent (e.g., `svchost.exe -k netsvcs -p -s Schedule` or `explorer.exe`).
- `data.win.eventdata.commandLine`: Exact runtime flags (e.g., `/autoclean /d C:`).
- `data.win.eventdata.hashes`: Ingests `MD5`, `SHA256`, and `IMPHASH` signatures.
- `data.win.eventdata.processGuid` & `processId`: Hex-encoded OS GUIDs and Process IDs used for process tracking.

2. Sysmon Event ID 3 (Network Connection)
Correlates internal OS processes with external network sockets:

- `data.win.eventdata.destinationIp`: Target IP address (e.g., Lumma C2 node `40.91.108.115`).
- `data.win.eventdata.destinationPort`: Target port (e.g., `443` HTTPS).
- `data.win.eventdata.image`: Binary responsible for opening the raw socket.

3. Sysmon Event ID 11 (File Create)
Monitors files written to disk, identifying staging operations in temporary or sensitive locations:

- `data.win.eventdata.targetFilename`: Destination file path (e.g., `C:\Users\ADMINI~1\AppData\Local\Temp\...\AssocProvider.dll` or Start Menu shortcuts).
- `data.win.eventdata.ruleName`: Mapped MITRE ATT&CK technique tags (e.g., `T1105` Ingress Tool Transfer, `T1023` Shortcut Modification).

4. Sysmon Event ID 13 (Registry Value Set)
Monitors Windows Registry modifications:

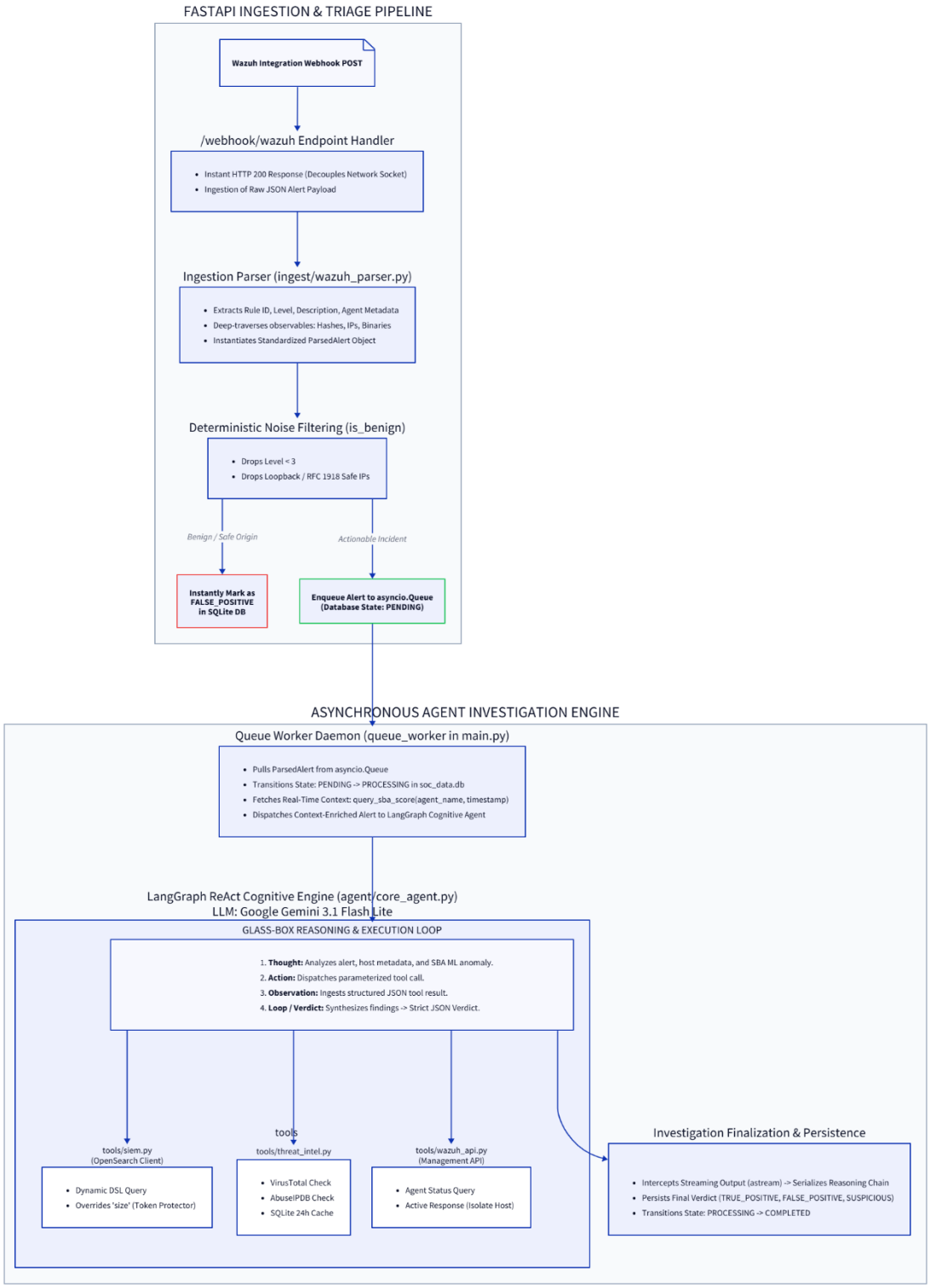
- `data.win.eventdata.targetObject`: Hive and key altered (e.g., `HKCR\WinRAR.ZIP\shellex\ContextMenuHandlers` or Inventory Application Version records).
- `data.win.eventdata.details`: Exact string/dword written into the registry key.

5. Windows Security & Application Channels

- **Event ID 4624 (Logon):** Ingests security logon attempts, logon types (Type 2 Interactive, Type 3 Network, Type 10 RemoteInteractive RDP), and user SIDs.
- **Event ID 4672 (Special Privileges Assigned):** Detects administrator logon privilege escalations.
- **Event ID 16384 (Microsoft-Windows-Security-SPP):** Standard Software Protection Platform background engine scheduling events.

3. Autonomous SOC Application Architecture & Cognitive Agent Workflow

The core autonomous analysis platform is built on an asynchronous Python micro-monolith framework. It integrates ingestion, queuing, LLM reasoning, historical context correlation, threat intelligence enrichment, and human-in-the-loop dashboarding.



3.1 Webhook Ingestion & Observable Parsing Pipeline

The platform entry point is an asynchronous HTTP POST route (`/webhook/wazuh`) exposed via FastAPI:

1. Decoupled Webhook Handshake
When the Wazuh integration daemon forwards an alert, FastAPI immediately acknowledges the payload with an HTTP 200 status code. This design pattern frees the Wazuh daemon's network socket, preventing backpressure on the SIEM even under heavy alert volumes.

2. Deep Observable Extraction (`ingest/wazuh_parser.py`)
The raw Wazuh JSON payload is passed to an observable parser that flattens nested Sysmon, network, and Windows event hierarchies into a normalized `ParsedAlert` data class:

- **Metadata:** `alert_id`, `timestamp`, `rule_id`, `rule_level`, `rule_description`, `agent_id`, `agent_name`, `agent_ip`.
- **Network Entities:** Source IP (`srcip`), Destination IP (`dstip`), Source Port, Destination Port.
- **Cryptographic Signatures:** Hashes extracted from Sysmon Event ID 1 (`MD5`, `SHA256`, `IMPHASH`) and Wazuh File Integrity Monitoring (FIM) fields (`md5_after`, `sha256_after`).
- **Execution Artifacts:** Process binaries (`image`), parent images (`parentImage`), command lines, target file paths, and registry keys.

3. Deterministic Benign Filter (`is_benign`)
To optimize LLM token usage and prevent API rate-limiting, incoming events are evaluated against deterministic triage rules before AI invocation:

- **Informational Noise Suppression:** Alerts with a rule level < 3 are dropped.
- **Trusted Origin Filters:** Network events originating exclusively from loopback (`127.0.0.1`, `::1`) or link-local subnets are dropped.
- **Automated Logging:** Dropped events are written directly to the database with a `FALSE_POSITIVE` verdict, requiring zero LLM inference time or cost.

3.2 Asynchronous Queuing & State Recovery (`asyncio.Queue`)
Events that pass deterministic triage are placed into an in-memory queue (`asyncio.Queue`) managed by a dedicated background worker (`queue_worker` in `main.py`).

Worker Processing Lifecycle

1. **State Transition:** The worker dequeues an alert and transitions its database record from `PENDING` to `PROCESSING`.
2. **Situational Awareness Enrichment:** Before invoking the LLM, the worker queries OpenSearch for the host's recent System Behavior Analytics (SBA) score.
3. **Agent Delegation:** The worker executes the LangGraph agent, passing the enriched `ParsedAlert` payload.
4. **Finalization:** The worker parses the agent's structured JSON verdict, updates the SQLite database with the decision, confidence score, summary, and reasoning chain, and marks the record as `COMPLETED`.

Startup Crash Recovery Routine

If the application server restarts while alerts are being processed, a startup routine queries `soc_data.db` for records stuck in `PENDING` or `PROCESSING` states. These alerts are automatically re-enqueued, ensuring no telemetry is lost during service interruptions.

3.3 The AI Reasoning Engine: LangGraph ReAct Architecture

The platform's cognitive reasoning core (`agent/core_agent.py`) uses a LangGraph ReAct (Reasoning and Acting) framework, powered by **Google Gemini 3.1 Flash Lite** via `ChatGoogleGenerativeAI`.

The ReAct Execution Cycle

Rather than using single-shot prompting, the agent operates in an iterative investigation loop:

$$\text{Thought} \longrightarrow \text{Action (Tool Call)} \longrightarrow \text{Observation (Tool Output)} \longrightarrow \text{Next Step}$$

1. **Thought:** The agent analyzes the alert context, identifies missing evidence, and forms a testable hypothesis.
2. **Action:** The agent executes a tool (e.g., querying historical SIEM logs or checking threat intelligence).
3. **Observation:** The agent evaluates the structured JSON response returned by the tool.
4. **Synthesis:** The agent continues investigating until it has sufficient evidence to output a final structured JSON decision.

Glass-Box Transparency & State Interception

A core capability of the platform is **Glass-Box Transparency**. By intercepting LangGraph's streaming output (`astream`), the application categorizes and captures every stage of the agent's reasoning process:

- Standard AI messages are recorded as **Thoughts**.
- Tool invocations and their parameters are recorded as **Actions**.
- Tool outputs are recorded as **Observations**.

This sequence is serialized into a `reasoning_chain` JSON array, providing human analysts with a complete audit trail for every automated decision.

3.4 Decoupled Tool Integrations & Telemetry Access

The agent interacts with the environment through modular tools defined via LangChain's `@tool` decorator.

1. Dynamic SIEM Log Retrieval (`tools/siem.py`)

Provides real-time access to the Wazuh Indexer (OpenSearch) using an asynchronous client (`AsyncOpenSearch`):

- **Dynamic Query Generation:** The LLM generates OpenSearch Domain Specific Language (DSL) JSON queries to locate historical evidence (e.g., matching `processId`, parent processes, or network events within specific time ranges).
- **Token Protection (Query Limiting):** To prevent large query responses from overflowing the LLM's context window, the tool automatically overrides the query's

`size` parameter (capping returned events at 10) while preserving aggregate hit counts (`total_hits`).

2. Threat Intelligence Enrichment (`tools/threat_intel.py`)
Enriches indicators of compromise (IoCs) extracted from alerts:

- **VirusTotal API:** Queries file hashes (`MD5`, `SHA256`) to retrieve detection ratios and community reputation scores.
- **AbuseIPDB API:** Queries external IP addresses to retrieve confidence-of-abuse scores, historical attack reports, and country of origin.

3. Rate-Limiting Mitigation & Persistent Cache (`tools/cache.py`)
External threat intelligence APIs often have strict rate limits. To mitigate this, a local SQLite caching layer (`soc_cache.db`) stores IoC lookup results:

- **Key-Value Store:** Maps queried hashes and IP addresses directly to their JSON API responses.
- **Time-to-Live (TTL):** Cached results remain valid for 24 hours. Subsequent queries for the same IoC within this window return instantly from local storage, bypassing external API calls.

4. Active Response & Containment (`tools/wazuh_api.py`)
Enables the agent to execute containment actions directly on endpoints using the Wazuh Manager REST API (port `55000/TCP`):

- **Authentication:** Handles dynamic JSON Web Token (JWT) acquisition and renewal.
- **Active Response Actions:** Allows the agent to query agent health, restart services, or execute host isolation routines (`ISOLATE_HOST`) to contain confirmed threats.

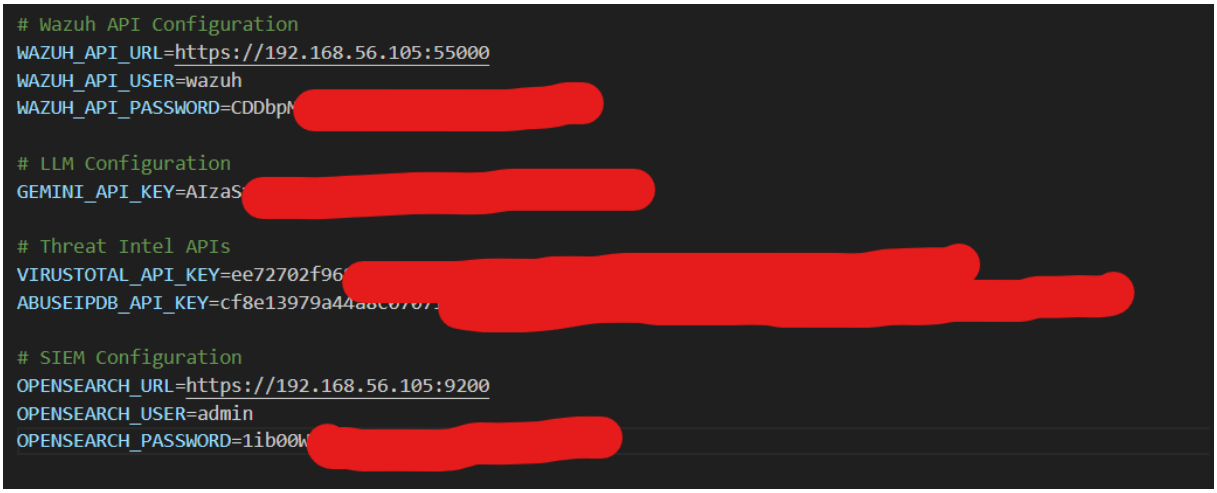
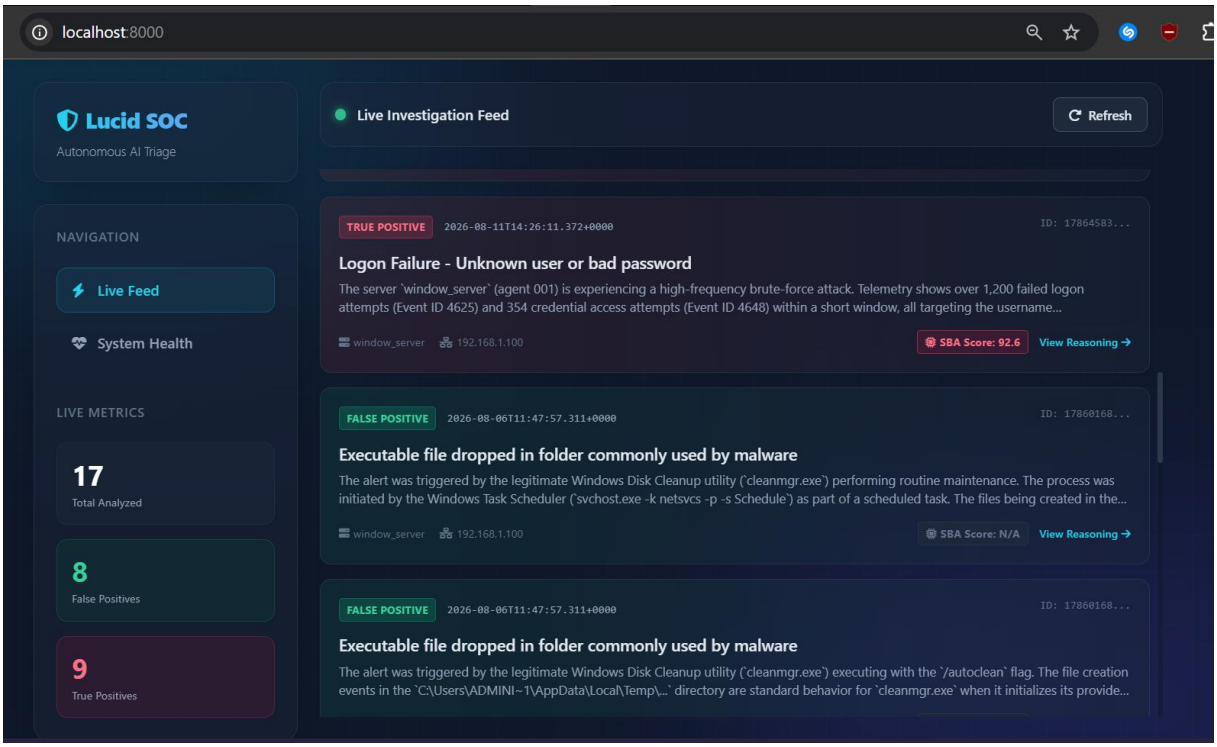


Figure 10 env file code snippet



4. System Behavior Analytics (SBA): Machine Learning Architecture

The System Behavior Analytics (SBA) microservice provides unsupervised anomaly detection across aggregated endpoint telemetry. Instead of relying on static, rule-based signatures, SBA evaluates the volume and behavioral patterns of unstructured system logs over sliding time windows.

SBA OFFLINE TRAINING PIPELINE

Historical Raw Telemetry
(wazuh-archives-*) - 14 Days OpenSearch Ingestion

Drain3 Streaming Template Mining (`train.py`)

- Replaces Dynamic Variables (IPs, Paths, Hex) -> Wildcard Tokens `<*>`
- Groups Unstructured Logs into Static Semantic Template Clusters (0 to N)
- Freezes Vocabulary State -> Serializes to disk: `models/drain3_state.bin`

Time-Series Feature Vectorization & Sliding Window Matrix Generation

- Groups Logs by host -> Aggregates Frequencies into 5-Minute Mathematical Buckets
- Generates Feature Matrix **X**: Rows = Time Intervals, Columns = Drain3 Clusters

Feedback Injection Loop (Human-in-the-Loop "Mark as Normal")

- Ingests User-Marked Timestamps from SQLite `soc_data.db` (`sba_feedback` table)
- Maps Timestamp to 5-Min Floor Bucket -> Locates Exact Feature Vector
- Duplicates Feature Vector 50x in Training Set (Forced High-Density Node)

Synthetic Outlier Injection & Isolation Forest Model Fitting (PyOD IForest)

- Injects Outlier Vector: Massive Count in Unseen Template (`cluster_0`)
- Fits Tree Ensembles: Isolate Anomalies via Scarcity and Short Path Lengths

ECDF Calibration & Serialization

- Calibrates Raw Output Scores -> Strictly Scales to 0..100 Human/AI Percentile Range
- Serializes Fitted Model, Scaler & Column Definitions using joblib into `./models`

Hot-Reload Volume Mount
(./models -> /app/models)

SBA LIVE INFERENCE PIPELINE

Continuous Inference Daemon (`run_scheduler.py` - Every 5 Min)

- Queries OpenSearch: Fetches Last 5 Minutes of Telemetry for Active Endpoints

PyOD Isolation Forest Model Scoring & Percentile Scaling

- Computes Decision Score -> Maps Through ECDF -> Generates `ml_sba_score` (0-100)

Anomaly Evaluation

Anomaly Evaluation

Score <= 75
(Normal)

Score > 75
(Anomalous)

Explainable AI: SHAP TreeExplainer

- Calculates Marginal Feature Attribution Weights pushing towards anomaly
- Translates Top Feature Indices -> Drain3 Cluster Definitions

OpenSearch Telemetry Push (`soar_bridge_snippet.py`)

- Writes Document to Dedicated Index: `sba-host-scores`
- Payload: `ml_sba_score` , `anomaly_severity` , `anomaly_nature` , `sba_contributing_factors`

4.1 Unstructured Log Parsing: Drain3 Template Mining

Transforming high-volume, variable text strings into structured feature vectors is a primary challenge in automated log analysis. SBA addresses this using **Drain3**, an online log parsing algorithm that operates via a parse-tree structure.

1. **Tokenization and Masking:** Ingests raw strings (e.g., Windows Sysmon events, Security-SPP notifications) and replaces variable tokens—such as IP addresses, file paths, GUIDs, and timestamps—with wildcards (`<*>`).
2. **Template Clustering:** Groups structurally identical messages into persistent cluster IDs:
    - Raw: `Software Protection service scheduled successfully for restart at 2026-12-18T14:33:04Z. Reason: RulesEngine.`
    - Template: `Software Protection service scheduled successfully for restart at <*>. Reason: <*>.`
3. **Vocabulary Serialization:** Freezes and serializes the global template vocabulary to disk (`models/drain3_state.bin`), establishing a consistent feature space for downstream machine learning models.

4.2 Time-Series Feature Vectorization

SBA evaluates host behavior over time rather than analyzing individual log lines in isolation:

- **Sliding Windows:** Ingested logs are grouped by `host` and segmented into non-overlapping **5-minute sliding windows**.
- **Frequency Vector Construction:** For each 5-minute interval, the microservice counts the occurrences of each Drain3 template cluster.
- **Feature Space:** Represents host activity during window t as a vector:

$$x_t = [c_0, c_1, c_2, \dots, c_m] \in R^m$$

where c_i denotes the frequency of log template cluster i within that time window.

4.3 Unsupervised Anomaly Detection: PyOD Isolation Forest

The vector matrix is evaluated using an **Isolation Forest (**`IForest`**)** model from the Python Outlier Detection (PyOD) library.

Why Isolation Forests?

Unlike profiling models that attempt to define all variations of "normal" behavior, Isolation Forests isolate anomalies directly. Because anomalous system behavior typically occurs infrequently and exhibits unusual feature values, anomalies are isolated near the root of the tree ensemble (yielding short average path lengths $h(x)$).

Synthetic Outlier Injection

To ensure zero-day attacks or previously unseen behaviors are reliably penalized, a synthetic data point is injected during training. This point contains an extreme frequency spike in the unknown log cluster (`cluster_0`), forcing the model to assign high anomaly scores to novel, unrecognized system events.

Empirical CDF Score Calibration
Raw decision scores from tree ensembles are mathematically unbounded. To convert these into an intuitive scale for human analysts and AI agents, an **Empirical Cumulative Distribution Function (ECDF)** calibrates the output against the baseline training distribution:

$$\text{Score}(x) = \text{ECDF}\big(\text{RawScore}(x)\big) \times 100 \in [0{,}100]$$

This normalizes the final `ml_sba_score` into a percentile range:

- `0 - 50`: Baseline / Normal Activity.
- `51 - 75`: Elevated Background Noise.
- `76 - 100`: High / Critical Behavioral Anomaly.

4.4 Explainable AI (XAI): SHAP Attribution Engine

When a host's anomaly score exceeds the operational threshold (`ml_sba_score > 75`), the microservice identifies the underlying drivers using **SHAP (SHapley Additive exPlanations)**:

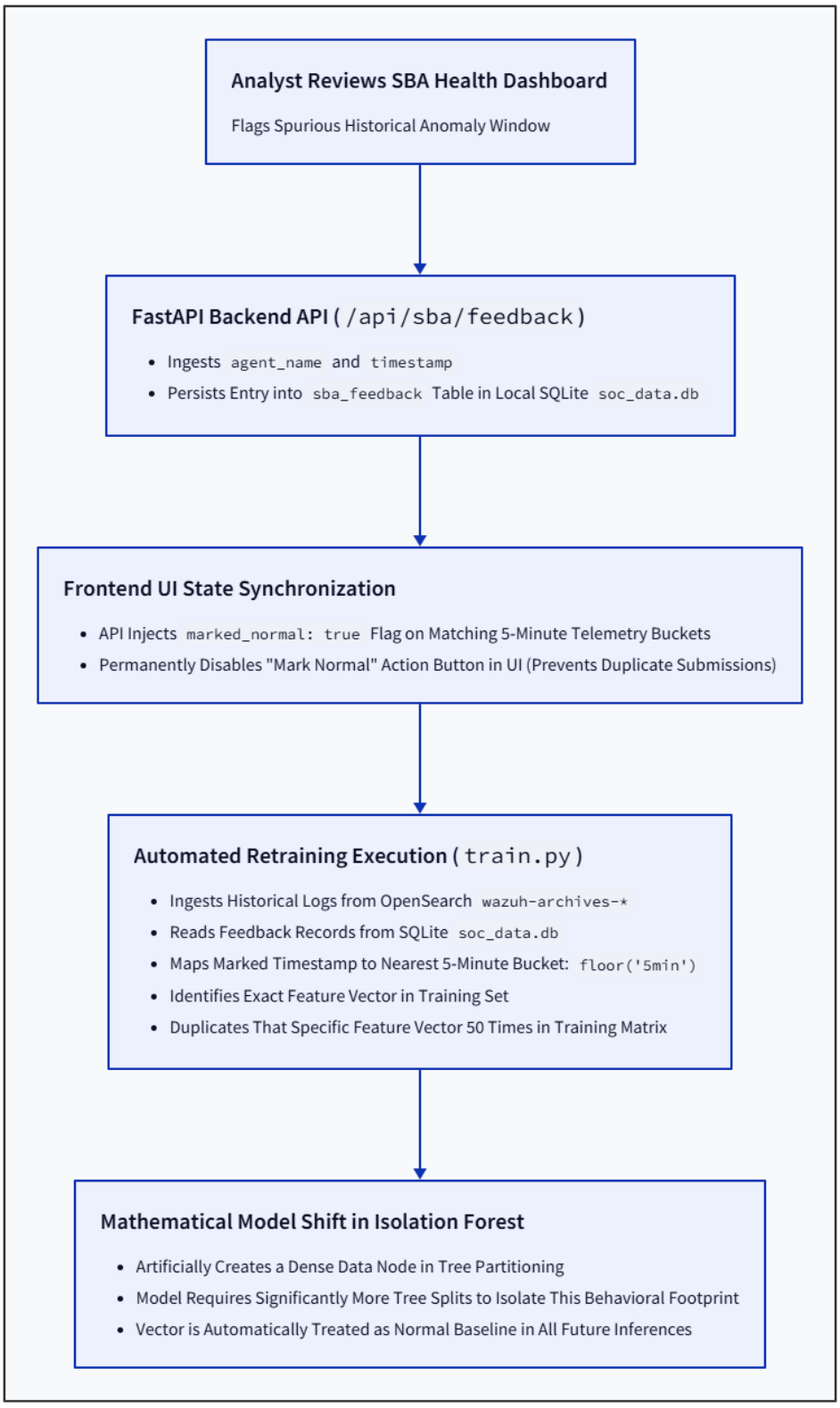
$$\phi_i(x) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

1. **TreeExplainer Integration:** A `shap.TreeExplainer` calculates the exact marginal contribution (ϕ_i) of each log template feature to the anomalous score.
2. **Feature Mapping:** The top contributing feature indices are mapped back to their human-readable Drain3 template definitions.
3. **Structured Attribution Output:** Rather than reporting an opaque mathematical anomaly, the system generates actionable explanations (e.g., `"Windows Event 7036 (Service Control Manager) [Weight: 0.229]"` or `"Executable dropped in Windows root folder [Weight: 0.348]"`).

4.5 Closed-Loop Feedback Architecture: "Mark as Normal"

To continuously adapt to benign environment changes and minimize recurring false positives, the platform implements a human-in-the-loop feedback mechanism:

"MARK AS NORMAL" FEEDBACK LOOP



1. **State Persistence:** When an analyst flags a false anomaly via the UI, the FastAPI backend records the `agent_name` and `timestamp` in the `sba_feedback` table within `soc_data.db`.
2. **UI State Awareness:** When rendering the historical telemetry table, the API cross-references the feedback table and applies a `marked_normal: true` flag to matching intervals, rendering the corresponding UI button in a disabled green state.
3. **Training Injection (Data Duplication):** During scheduled retraining (`train.py`), the script reads the feedback table, rounds timestamps to their 5-minute bucket (`floor('5min')`), and duplicates matching feature vectors **50 times** in the training matrix.
4. **Mathematical Impact:** Because Isolation Forests identify anomalies based on data scarcity and short tree paths, artificially clustering 50 identical vectors creates a dense feature node. The model requires significantly more random splits to isolate these points, establishing this specific behavioral pattern as part of the normal host baseline in future inference cycles.

4.6 Containerization & Zero-Downtime Hot-Reloading

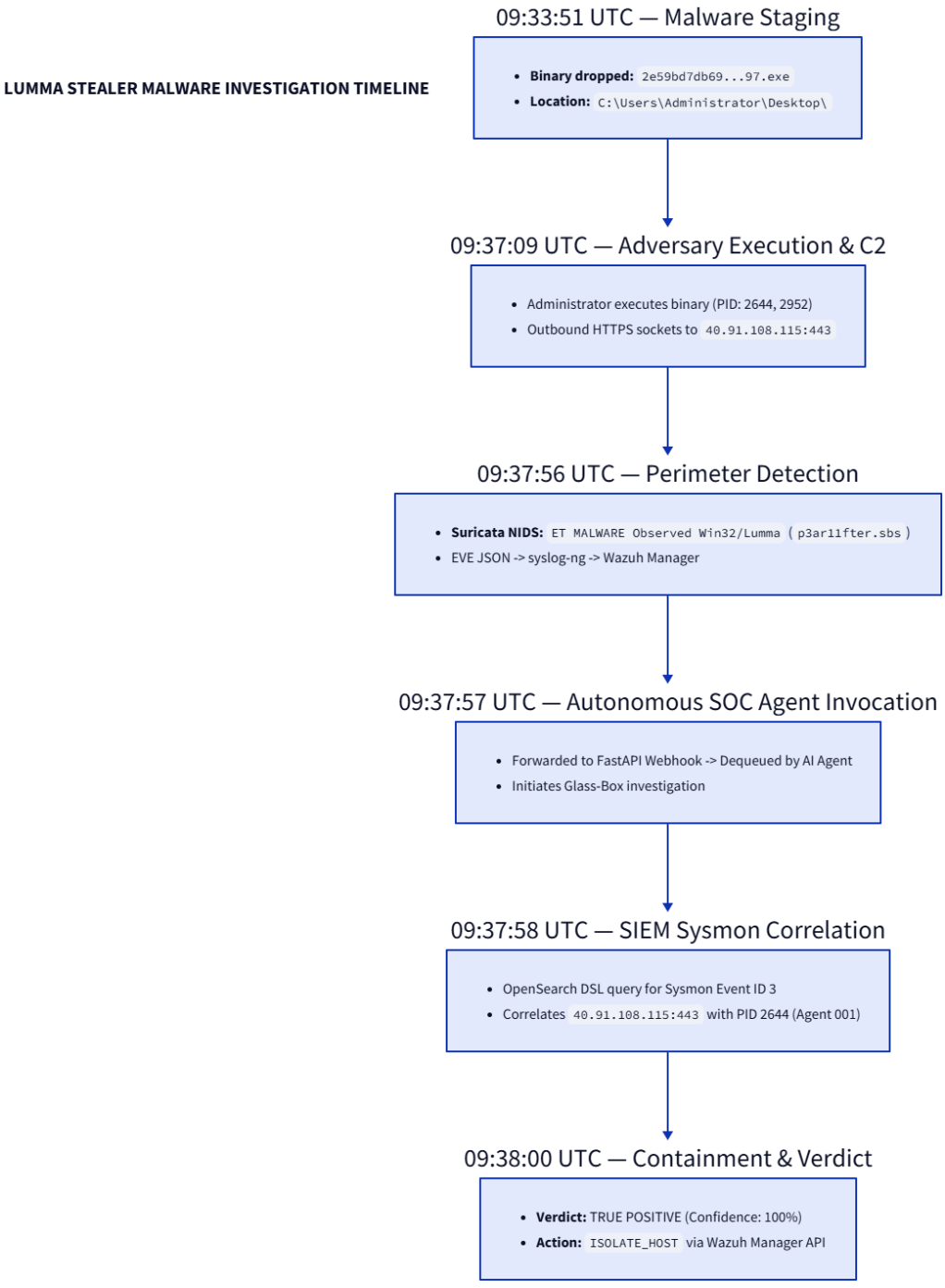
The SBA microservice runs as an independent container managed via Docker Compose:

- **Configuration:** Built on `python:3.10-slim`, executing `run_scheduler.py` to maintain the continuous 5-minute inference loop.
- **Network Attachment:** Attaches to the shared `soc_network`, providing direct access to OpenSearch (`192.168.1.101:9200`) and the FastAPI backend.
- **Dynamic Model Mounting:** The host machine's `./models` directory is volume-mounted to `/app/models` inside the container. When retraining completes on the host, the container accesses updated model binaries on its next inference cycle, requiring zero downtime or container restarts.

5. Practical Investigation Deep-Dives & Real-World Validation

To validate the autonomous SOC platform, real-world attack scenarios and operational baseline events were executed in the lab environment. The agent's decision logic was evaluated against manual triage methodologies used by senior SOC analysts at Orange Cyberdefense, confirming its investigative steps and conclusions.

5.1 Deep Dive: Malware Execution (Lumma Stealer C2 Infection)

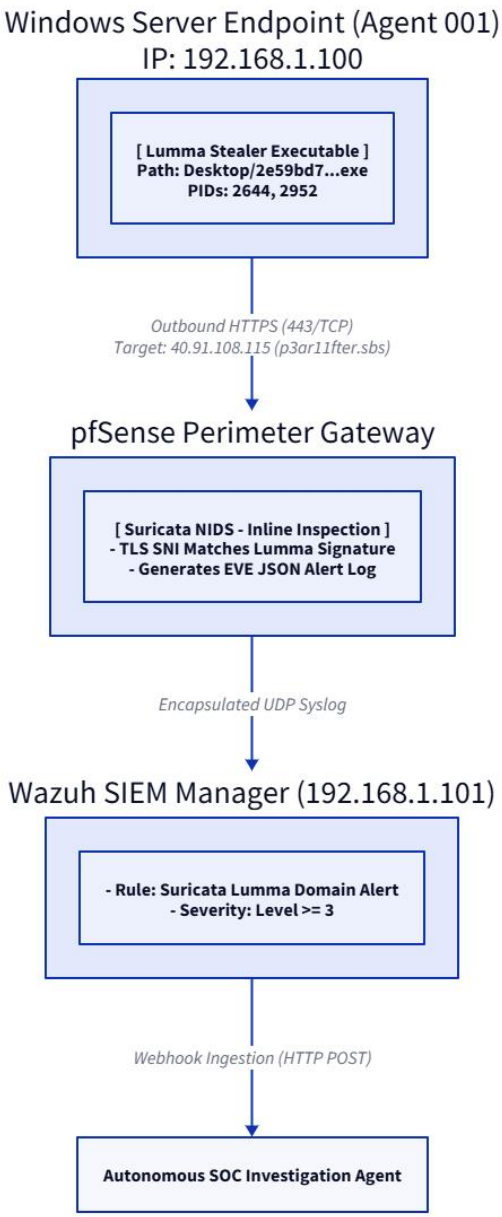


Incident Narrative & Telemetry Correlation

A live Lumma Stealer binary was retrieved from MalwareBazaar and executed on the Windows Server endpoint (`WIN-5RJ7UQ2G3GE` / Agent `001`) under the Administrator account:

Binary Path: $\texttt{C:\textbackslash Users\textbackslash Administrator\textbackslash Desktop\textbackslash 2e59bd7db699a8a7063c44ad2da160316941bef24b18654eae8de5fc97cabb57.exe}$

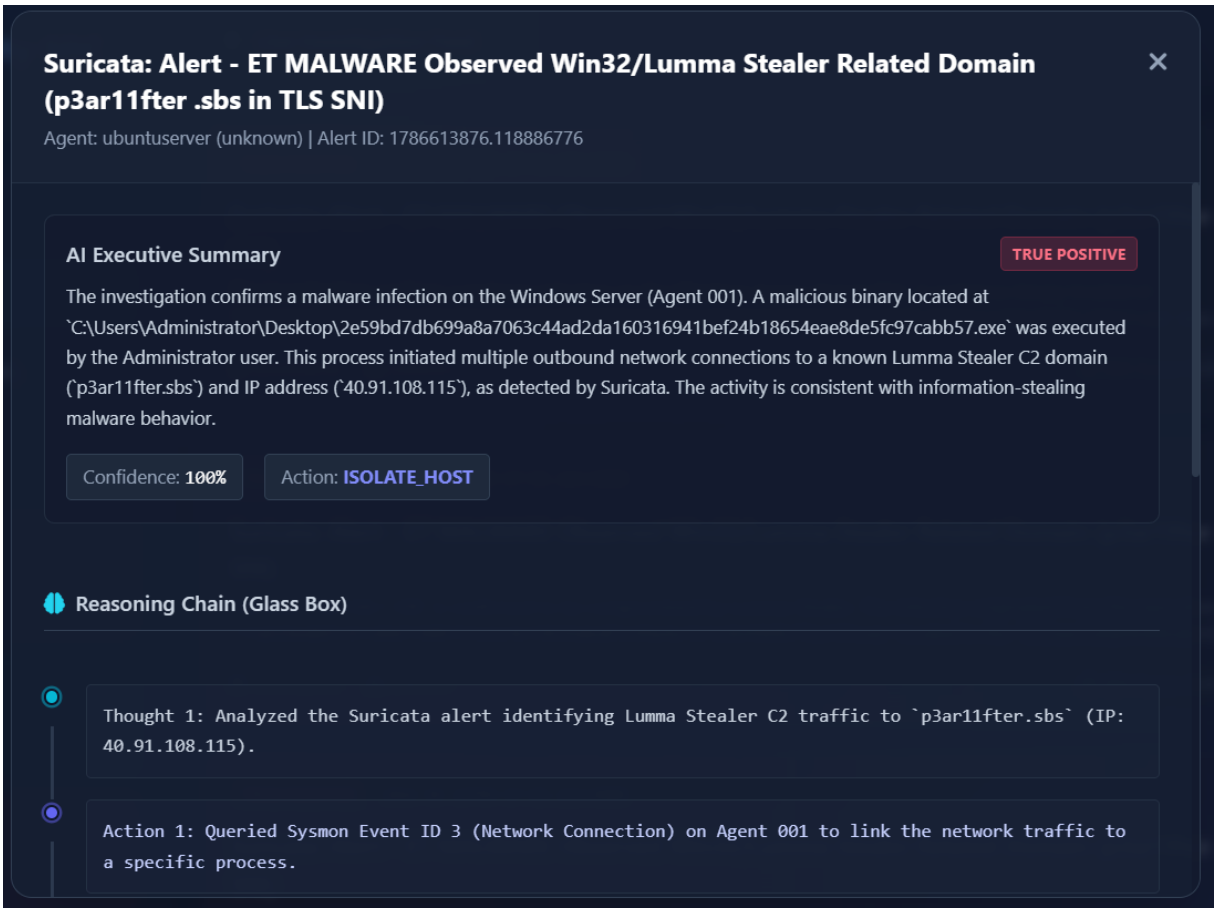
Upon execution, the process initiated encrypted outbound command-and-control (C2) connections over TLS to external endpoints.

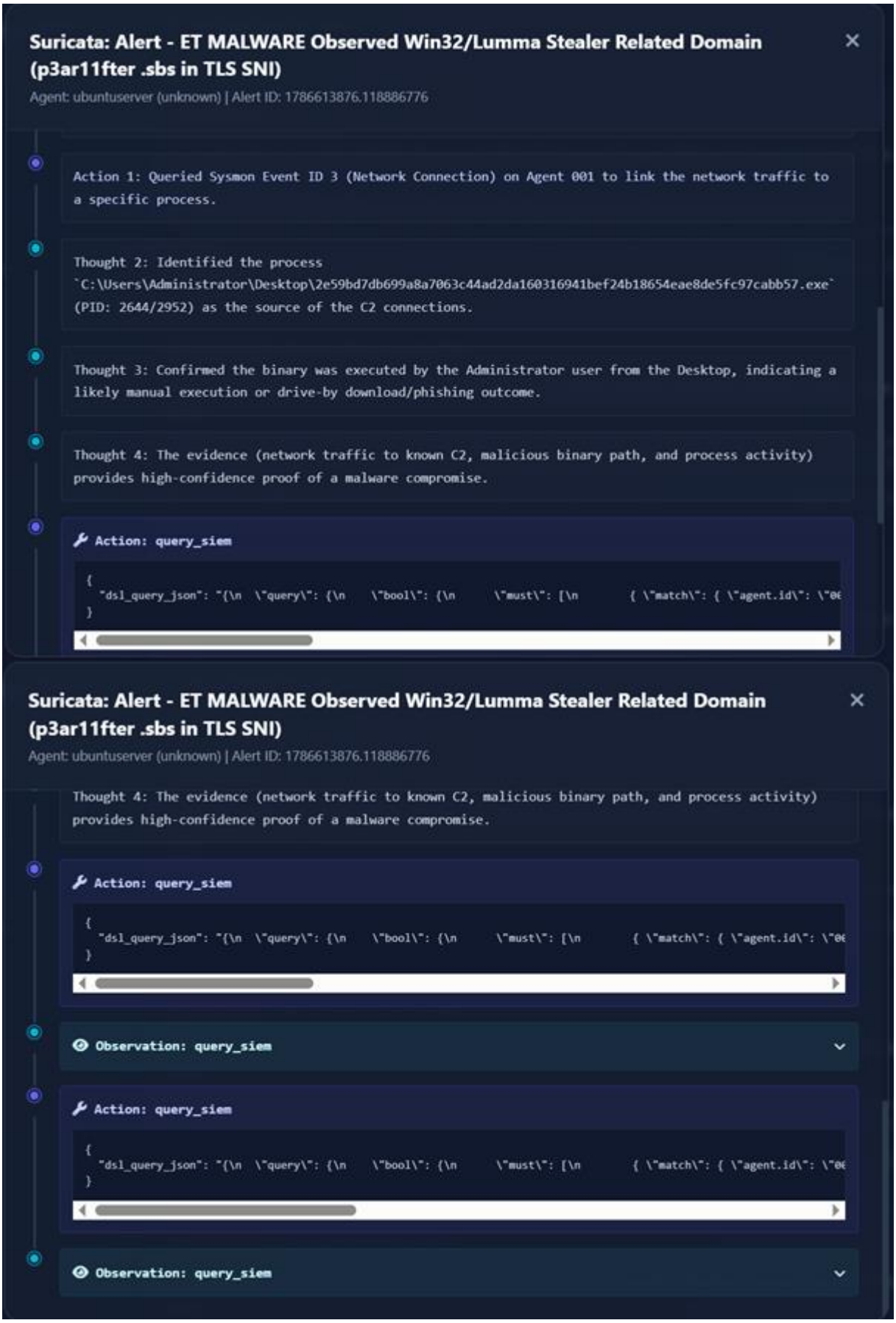


Network Detection: Suricata inspected the TLS handshake SNI on the `LAN (em1)` interface and triggered alert `ET MALWARE Observed Win32/Lumma Stealer Related Domain (p3ar11fter.sbs in TLS SNI)`.

1. **SIEM Ingestion:** The alert was forwarded via syslog-ng to the Wazuh Manager, which matched it against Suricata detection rules and transmitted an HTTP webhook payload to the Autonomous SOC platform.
2. **Autonomous Investigation:** The cognitive agent received the alert (`Alert ID: 1786613876.118886776`) and initiated an autonomous ReAct investigation loop.

Step-by-Step Glass-Box Agent Reasoning Analysis





Step 1: Ingestion & Initial Assessment

- `Thought 1`**:** *"Analyzed the Suricata alert identifying Lumma Stealer C2 traffic to* `p3ar11fter.sbs` *(IP:* `40.91.108.115`*)."*
- `Action 1`**:** The agent recognizes that while the firewall detected network-level C2 traffic, it must correlate the connection with a specific endpoint process. It constructs an OpenSearch DSL query targeting Sysmon Event ID 3 on Agent `001`.

Step 2: SIEM Query Construction & Historical Pivot
The agent formats and executes an OpenSearch DSL query via `tools/siem.py`:

```
{
  "query": {
    "bool": {
      "must": [
        { "match": { "agent.id": "001" } },
        { "match": { "data.win.system.eventID": 3 } },
        { "range": { "@timestamp": { "gte": "2026-08-13T09:30:00Z",
"lte": "2026-08-13T10:00:00Z" } } }
      ]
    }
  }
}
```

Step 3: Observation & Process Correlation

- `Observation 1`**:** OpenSearch returns 21 matching Sysmon records (`total_hits: 21`). The agent extracts key telemetry fields from the returned events:
    - Source Binary: `C:\Users\Administrator\Desktop\2e59bd7db699a8a7063c44ad2da160316941bef24b18654eae8de5fc97cabb57.exe`
    - Process IDs: `2644` and `2952`
    - Source IP: `192.168.1.100` (Ephemeral Ports: `49828`, `49823`, `49835`, `49830`, `49815`, `49829`)
    - Destination IP: `40.91.108.115:443` (HTTPS)
    - User Context: `WIN-5RJ7UQ2G3GE\Administrator`
    - Additional Associated Domains: `a23-223-99-235.deploy.static.akamaitechnologies.com`

Step 4: Contextual Synthesis & Decision

- `Thought 2`**:** *"Identified the process* `C:\Users\Administrator\Desktop\2e59bd7db699a8a7063c44ad2da160316941bef24b18654eae8de5fc97cabb57.exe` *(PID: 2644/2952) as the source of the C2 connections."*

- `Thought 3`**:** *"Confirmed the binary was executed by the Administrator user from the Desktop, indicating a likely manual execution or drive-by download/phishing outcome."*
- `Thought 4`**:** *"The evidence (network traffic to known C2, malicious binary path, and process activity) provides high-confidence proof of a malware compromise."*

Step 5: Final Output & Containment

The agent concludes the investigation and outputs its final structured decision:


```
{
  "verdict": "TRUE_POSITIVE",
  "confidence": "100%",
  "action": "ISOLATE_HOST",
  "summary": "The investigation confirms a malware infection on the Windows Server (Agent 001). A malicious binary located at C:\\Users\\Administrator\\Desktop\\2e59bd7...exe was executed by the Administrator user. This process initiated multiple outbound network connections to a known Lumma Stealer C2 domain (p3ar11fter.sbs) and IP address (40.91.108.115), as detected by Suricata. The activity is consistent with information-stealing malware behavior."
}
```


Orange Cyberdefense Methodology Validation

The agent's automated investigation was reviewed by tier-3 SOC Analyst at Orange Cyberdefense, validating several key analytical steps:

1. **Pivoting from Perimeter to Host Telemetry:** An alert from a network sensor (Suricata) identifies a suspicious external connection but does not identify the responsible endpoint binary. Querying host instrumentation (Sysmon Event ID 3) to identify the specific process ID and executable path is the standard procedure for confirming network indicators.
2. **Correlating User Context and Execution Location:** Confirming that the executable was launched from a user directory (`Desktop`) under an administrative account establishes intent and execution context, ruling out background system services.
3. **Decisive Active Containment:** Executing host isolation (`ISOLATE_HOST`) prevents infostealers from completing data exfiltration or deploying secondary payloads across internal networks.
4.

5.2 Deep Dive: Benign System Activity / False Positive Resolution

Security teams frequently encounter high volumes of false-positive alerts triggered by legitimate system administration utilities executing in unexpected directories.

Executable file dropped in folder commonly used by malware

Agent: window_server (192.168.1.100) | Alert ID: 1786016877.147419314

Thought 4: No malicious indicators (e.g., network connections, unusual parent, or suspicious command-line arguments) were found.

Action: query_siem

```
{
  "dsl_query_json": "{\n  \"query\": {\n    \"bool\": {\n      \"must\": [\n        { \"term\": { \"agent.id\": \"001
}
```

Observation: query_siem

Action: query_siem

```
{
  "dsl_query_json": "{\n  \"query\": {\n    \"bool\": {\n      \"must\": [\n        { \"term\": { \"agent.id\": \"001
}
```

Observation: query_siem

Executable file dropped in folder commonly used by malware

Agent: window_server (192.168.1.100) | Alert ID: 1786016877.147419314

AI Executive Summary

FALSE POSITIVE

The alert was triggered by the legitimate Windows Disk Cleanup utility (`cleanmgr.exe`) executing with the `/autoclean` flag. The file creation events in the `C:\Users\ADMINI~1\AppData\Local\Temp\...` directory are standard behavior for `cleanmgr.exe` when it initializes its provider DLLs to perform disk cleanup tasks. The process was spawned by `svchost.exe` (Schedule service), which is consistent with a scheduled maintenance task.

Confidence: **95%** Action: **CLOSE_TICKET**

Reasoning Chain (Glass Box)

Thought 1: Analyzed the alert and identified the process as `cleanmgr.exe` (Disk Cleanup).

Action 1: Queried SIEM for process creation events for `cleanmgr.exe` (PID 456) to verify its command line and parent process.

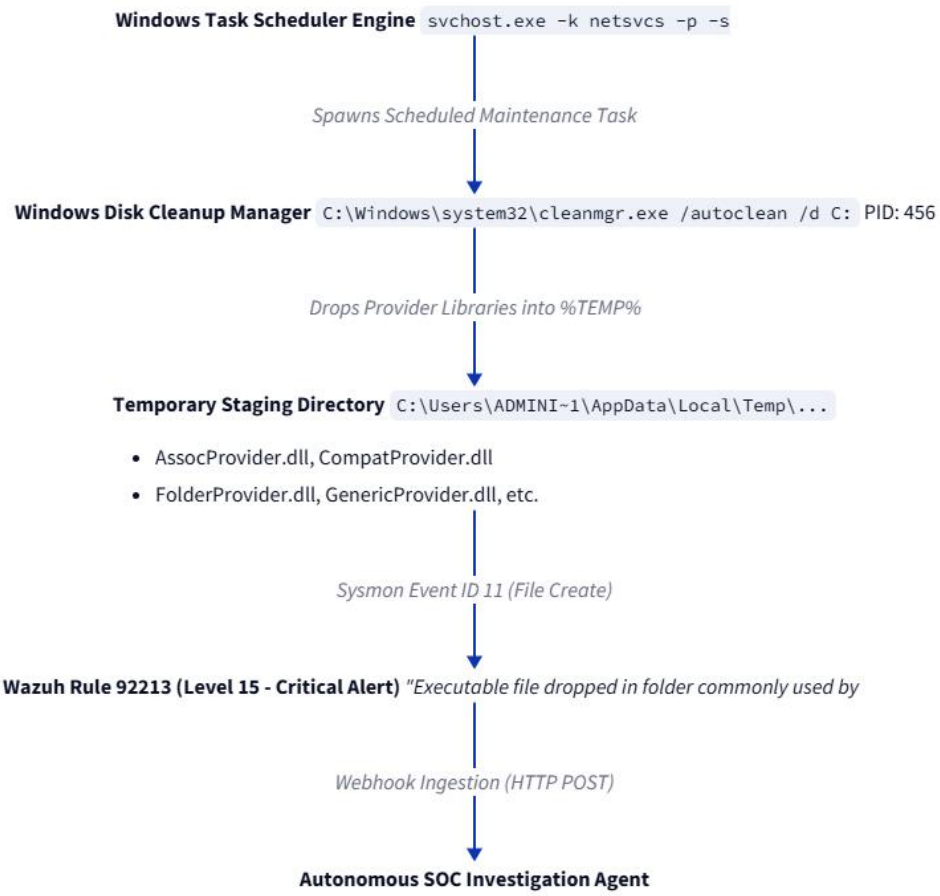
Thought 2: Confirmed `cleanmgr.exe` was executed with `C:\Windows\system32\cleanmgr.exe /autoclean /d C:` by `svchost.exe` (Schedule service).

Incident Context & Trigger Mechanism

Wazuh triggered a high-severity security alert (`Alert ID: 1786016877.147419314`, Rule `92213`, Severity Level `15`): "`Executable file dropped in folder commonly used by malware`" on Agent `001` (`192.168.1.100`).

The alert was triggered when multiple DLL files were written to a temporary user path:

$$\texttt{C:\Users\ADMINI\AppData\Local\Temp\4F497CBF-6916-4EE5-B708-3858DC3128BE\AssocProvider.dll}$$



Step-by-Step Glass-Box Agent Reasoning Analysis

Step 1: Alert Parsing & Process Identification

- `Thought 1`: *"Analyzed the alert and identified the process as* `cleanmgr.exe` *(Disk Cleanup)."*

- `Action 1`: The agent queries OpenSearch for the process creation event (Sysmon Event ID 1) associated with PID `456` to inspect its parent process and command-line arguments.

Step 2: OpenSearch DSL Execution
The agent constructs a focused DSL query via `tools/siem.py`:

```
{
  "query": {
    "bool": {
      "must": [
        { "term": { "agent.id": "001" } },
        { "term": { "data.win.system.eventID": "1" } },
        { "term": { "data.win.eventdata.processId": "456" } }
      ]
    }
  },
  "size": 15
}
```

Step 3: Observation & Parent Process Verification

- `Observation 1`: The SIEM returns matching process creation records:
  - Executable: `C:\Windows\System32\cleanmgr.exe`
  - Company: `Microsoft Corporation`
  - Command Line: `C:\Windows\system32\cleanmgr.exe /autoclean /d C:`
  - Parent Process: `C:\Windows\System32\svchost.exe`
  - Parent Command Line: `C:\Windows\system32\svchost.exe -k netsvcs -p -s Schedule`
  - User Context: `WIN-5RJ7UQ2G3GE\Administrator` (Parent User: `NT AUTHORITY\SYSTEM`)

Step 4: Contextual Evaluation

- `Thought 2`: *"Confirmed* `cleanmgr.exe` *was executed with* `C:\Windows\system32\cleanmgr.exe /autoclean /d C:` *by* `svchost.exe` *(Schedule service)."*
- `Thought 3`: *"The file creation events (DLLs in Temp) are standard behavior for* `cleanmgr.exe` *during its cleanup routine."*
- `Thought 4`: *"No malicious indicators (e.g., network connections, unusual parent, or suspicious command-line arguments) were found."*

Step 5: Final Automated Verdict & Ticket Closure
The agent completes its assessment and generates a structured closing verdict:

```
{
  "verdict": "FALSE_POSITIVE",
  "confidence": "95%",
  "action": "CLOSE_TICKET",
```

```
  "summary": "The alert was triggered by the legitimate Windows Disk Cleanup utility (cleanmgr.exe) executing with the /autoclean flag. The file creation events in the C:\\Users\\ADMINI~1\\AppData\\Local\\Temp\\... directory are standard behavior for cleanmgr.exe when it initializes its provider DLLs to perform disk cleanup tasks. The process was spawned by svchost.exe (Schedule service), which is consistent with a scheduled maintenance task."
}
```


5.3 Deep Dive: System Behavior Analytics (SBA) Macro Monitoring & Feedback Validation

In addition to granular per-alert triage, the platform continuously monitors macro-level endpoint behavior via the **System Health** interface (`health.html`).



Understanding Behavioral Classifications

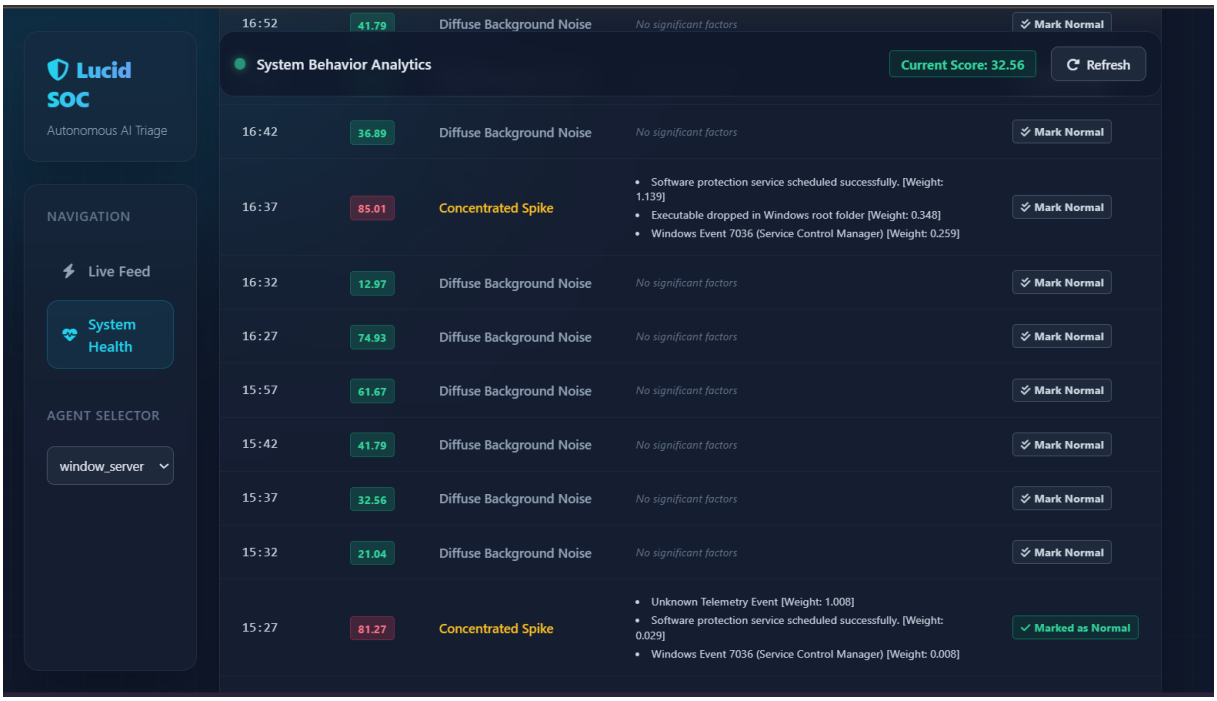
The SBA microservice evaluates the distribution of log templates across each 5-minute sliding window to categorize host behavior:

- `Diffuse Background Noise`**:** Indicates low-level, routine operational logs evenly distributed across common system services (e.g., standard background polling, keepalives, periodic scheduled tasks).
- `Concentrated Spike`**:** Indicates an unusual surge in a small number of specific log templates (e.g., high-frequency failed logons or rapid DLL drops in temporary folders).

SHAP Feature Attributions & Feedback Execution

The historical telemetry view exposes the exact log templates contributing to elevated anomaly scores:

1. **Baseline Anomaly Identification:** At timestamp `15:27 UTC`, the host registered an elevated score (`ml_sba_score: 83.11`, `Concentrated Spike`) driven by an unusual combination of system protection and service management events.
2. **Human Review & Feedback:** An analyst verified this activity as part of an authorized maintenance task and submitted feedback via the **"Mark Normal"** button.
3. **State Updates & Model Adaptation:**
    - The backend recorded the submission in the SQLite `sba_feedback` table.
    - The UI immediately updated the button state to a persistent, disabled **"✓ Marked as Normal"** indicator.
    - During the next retraining cycle (`train.py`), the corresponding feature vector was duplicated 50 times, incorporating that operational pattern into the baseline Isolation Forest model.



6. Summary Comparison: Traditional vs. Autonomous SOC Workflows

Operational Phase	Traditional SOC Triage	Autonomous Cognitive Platform	System Behavior Analytics (SBA)
Log Ingestion & Parsing	Static decoders, rigid schemas, manual regex maintenance.	Asynchronous webhooks normalize network, host, and Sysmon data on ingestion.	Drain3 dynamically groups unstructured logs into semantic template clusters.
Triage & Noise Filtering	Manual review of all alerts; high analyst fatigue from false alarms.	Deterministic filters drop informational and trusted-origin alerts immediately.	Percentile-scaled baseline scores contextualize overall host behavior.
Investigation Workflow	Analysts manually pivot across multiple consoles (SIEM, EDR, Firewall).	LangGraph ReAct agent queries SIEM and threat intel tools iteratively.	Evaluates aggregate telemetry patterns across sliding 5-minute windows.

Threat Intelligence	Manual copy-pasting of hashes and IPs into public threat lookups.	Automated API lookups with a 24-hour local SQLite caching layer.	Implicitly identifies novel behavioral deviations and zero-day spikes.
Auditability & Visibility	Static notes entered into ticketing systems; incomplete audit trails.	Glass-Box tracking captures full reasoning chains (Thoughts, Actions, Observations).	SHAP TreeExplainer calculates exact mathematical feature weights for anomalies.
Containment & Response	Manual intervention (e.g., initiating network isolation during business hours).	Confirmed true positives trigger automated active responses (`ISOLATE_HOST`).	Situational awareness context is appended to alerts before agent execution.
Continuous Learning	Manual, periodic rule-tuning and regex updates.	SQLite-persisted alert outcomes inform future investigation context.	"Mark as Normal" feedback adapts the Isolation Forest via vector duplication.